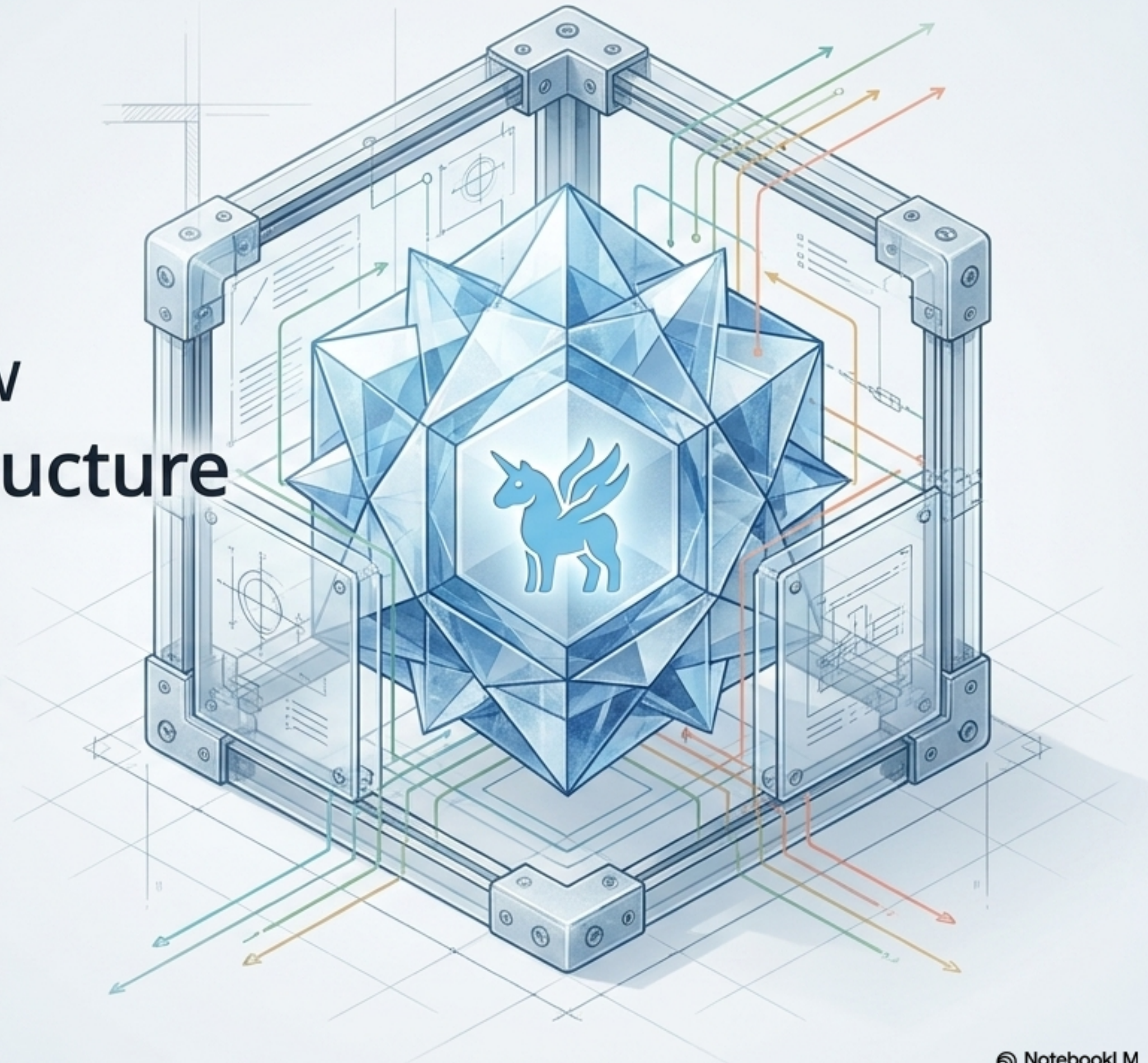
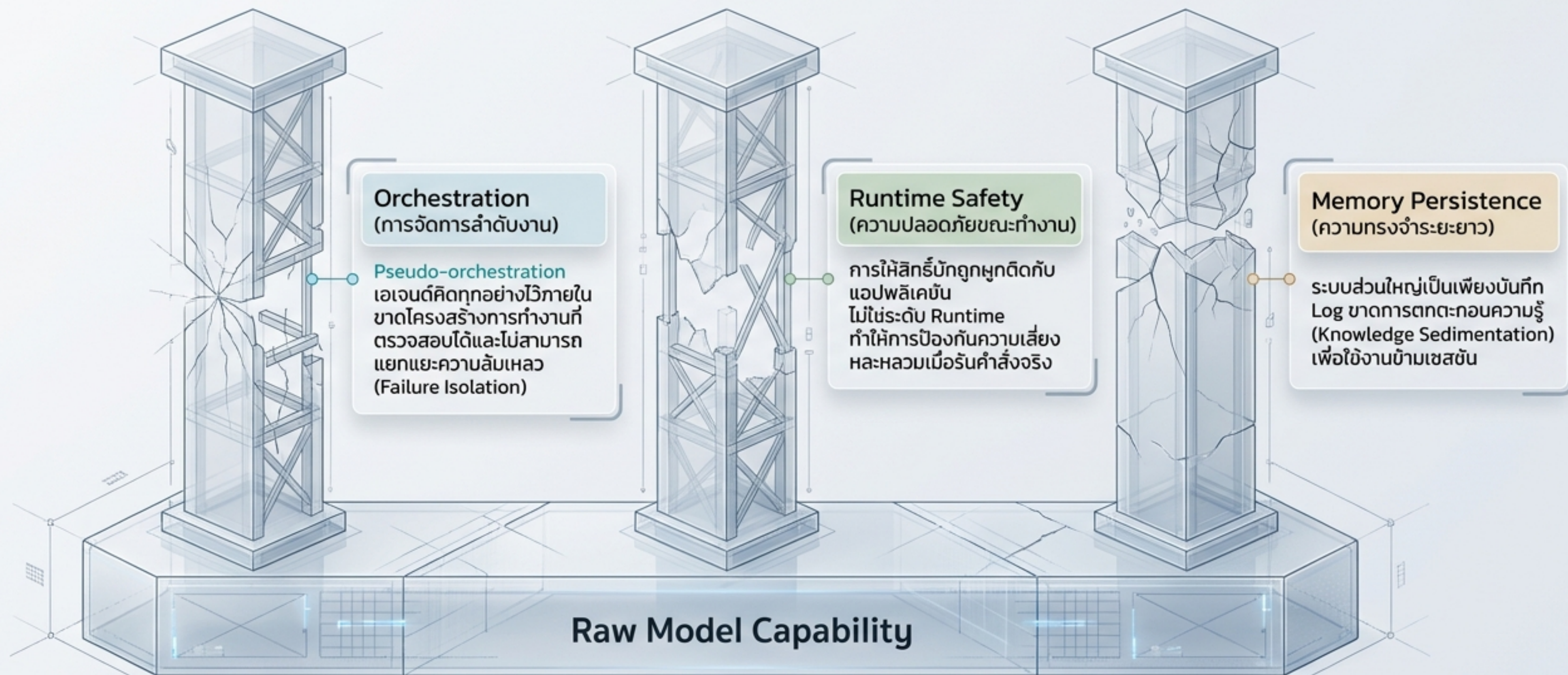


ก้าวข้าม Prompt Engineering: สถาปัตยกรรม SemaClaw สำหรับ Agentic Infrastructure ระดับ Production

การเปลี่ยนผ่านสู่ Harness Engineering
เพื่อควบคุม Orchestration, Safety และ
Memory ใน AI Agents

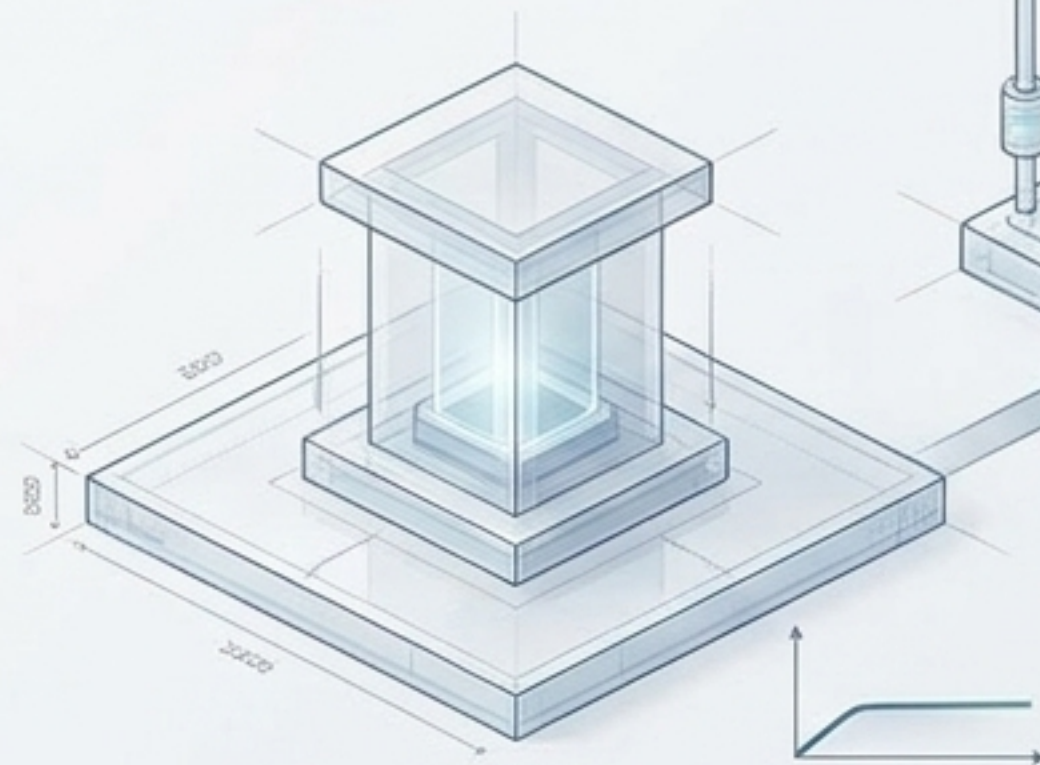


ขีดจำกัดของ Raw Model Capability ในโลกความจริง



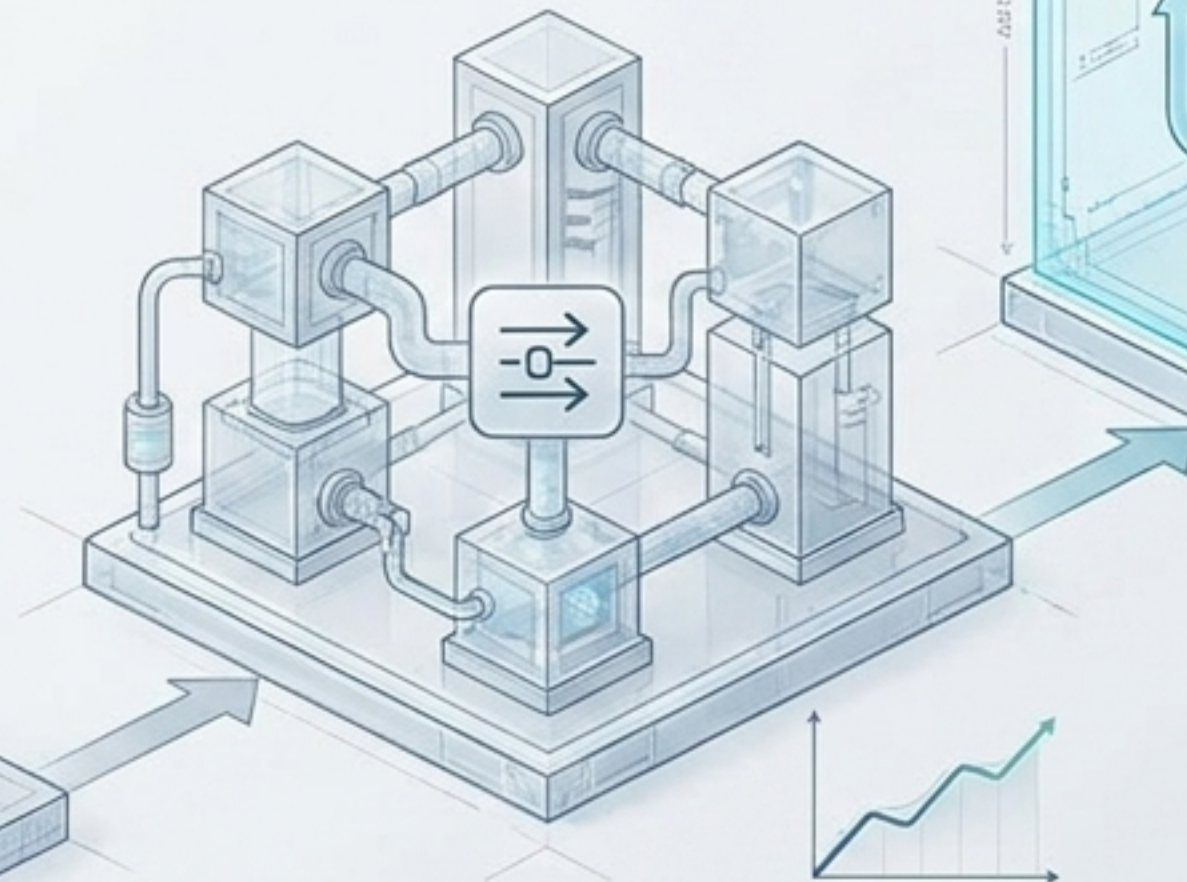
ปัญหาเหล่านี้ไม่สามารถแก้ได้ด้วยโมเดลที่ฉลาดขึ้น แต่ต้องแก้ด้วยระบบสภาพแวดล้อม (Harness Design)

วิวัฒนาการสู่ Harness Engineering



Phase 1: Prompt Engineering

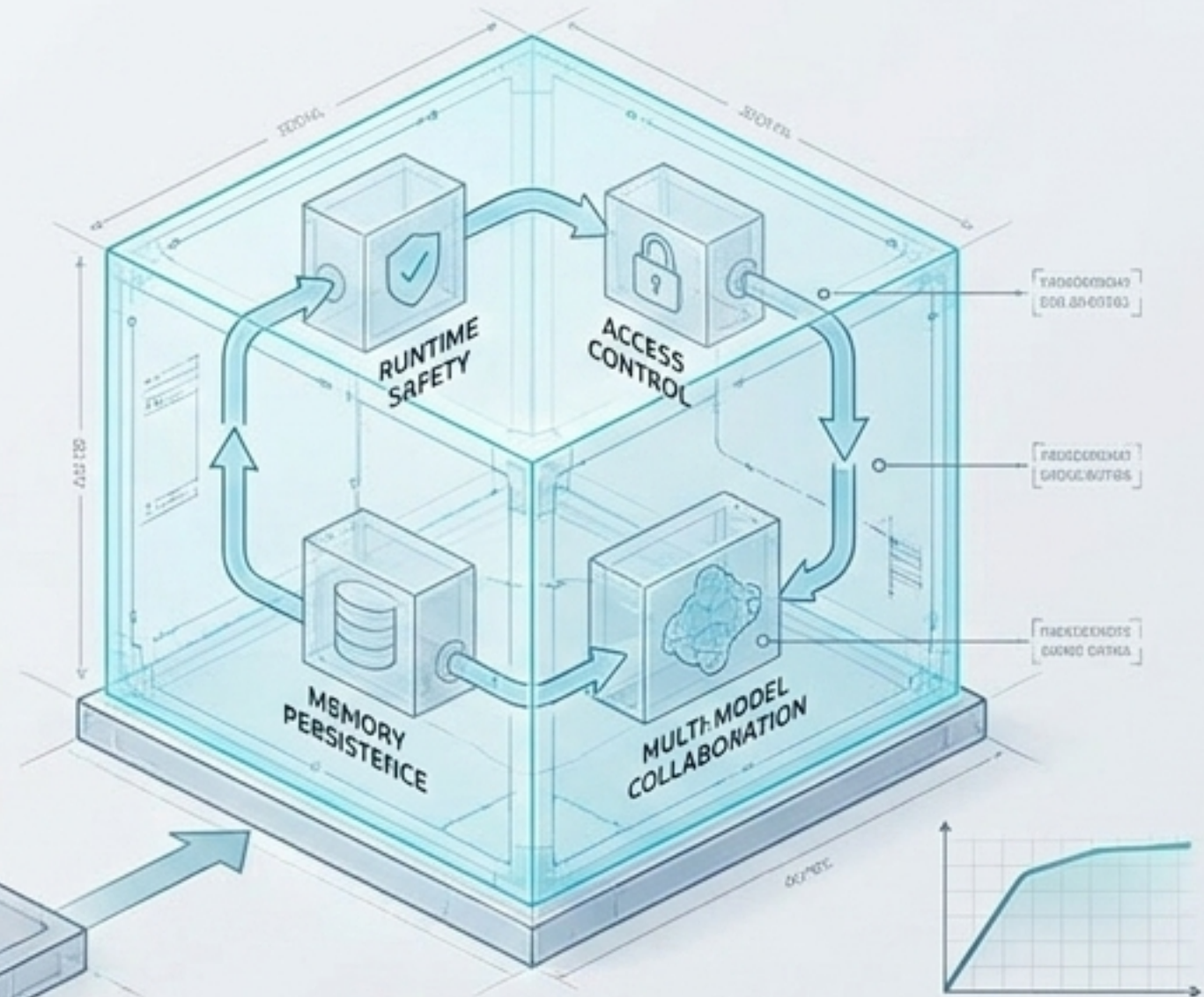
การควบคุมผ่านชุดคำสั่งเดี่ยว
(Single Inference)
โมเดลเป็นเพียงเครื่องสร้างข้อความ



Phase 2:

Context Engineering

การจัดการหน้าตาบริบท (RAG)
ป้อนข้อมูลที่ถูกต้องในเวลาที่เหมาะสม



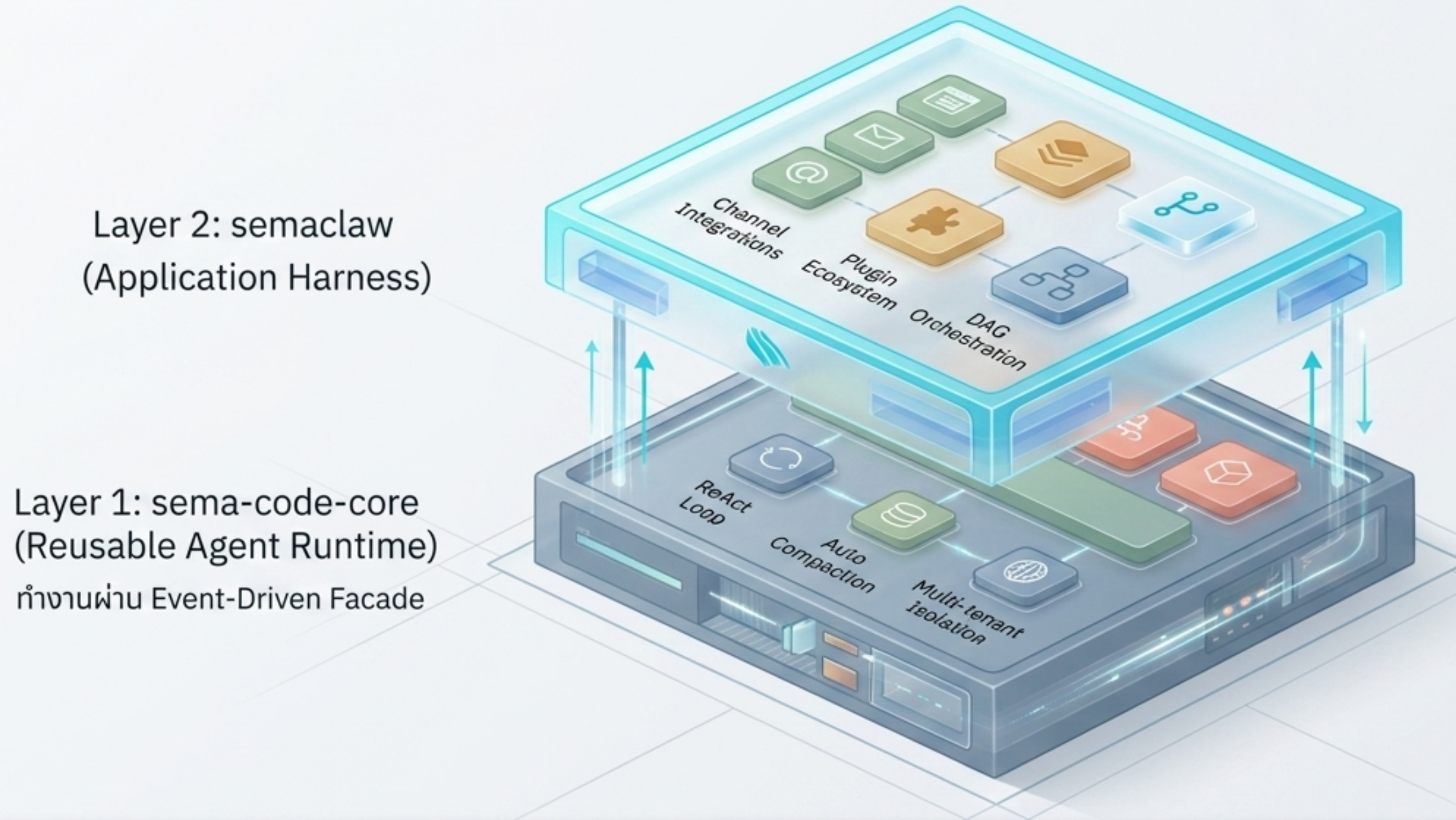
Phase 3: Harness Engineering

การออกแบบโครงสร้างพื้นฐานแบบวงจรปิด (Complete Infrastructure) ครอบคลุม Runtime, สิทธิการเข้าถึง, ความจำ และการทำงานร่วมกันหลายโมเดล



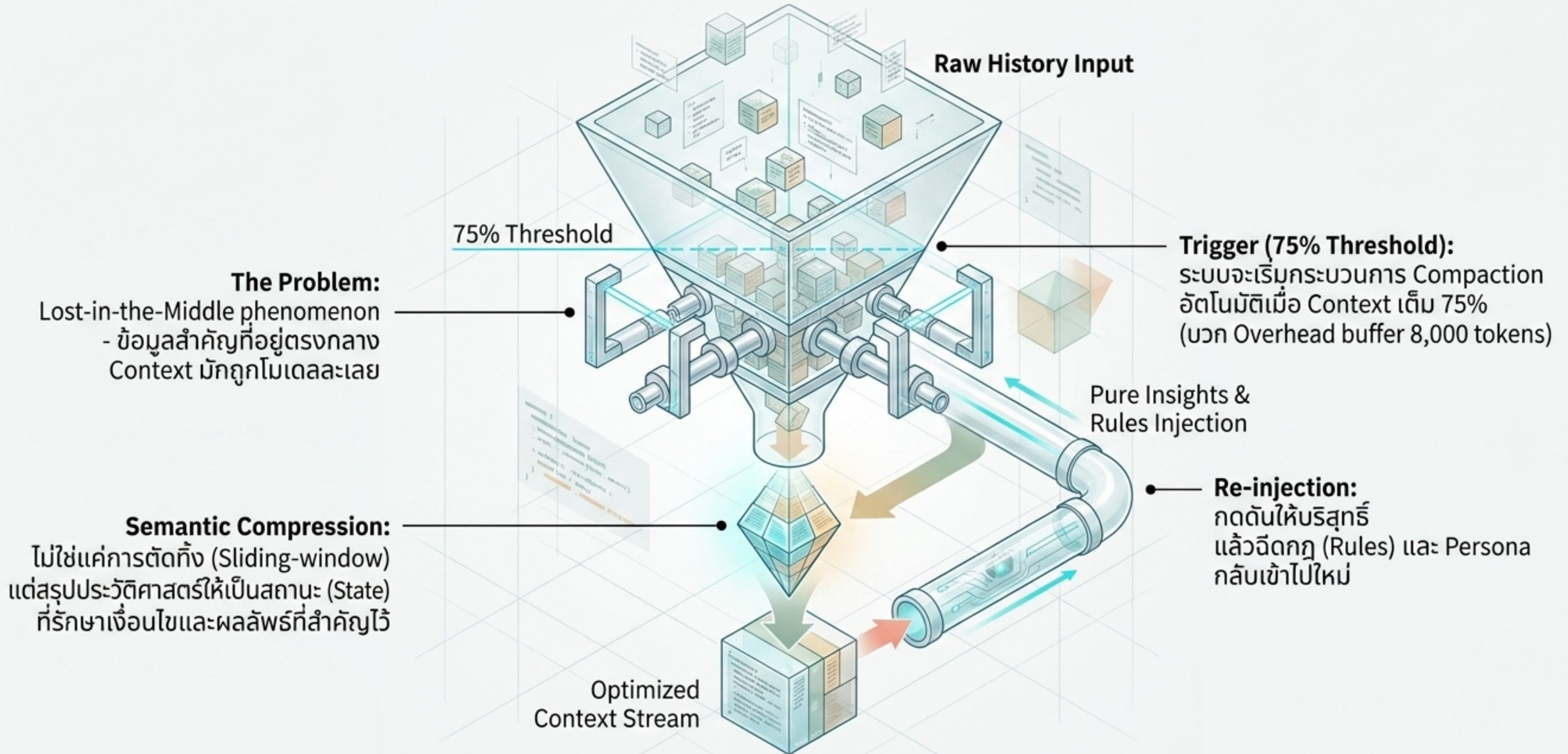
ข้อมูลเชิงประจักษ์จาก LangChain Terminal Bench 2.0 ชี้ว่า Harness Configuration ที่ดีสามารถเพิ่ม Task Completion ได้จาก 52.8% เป็น 66.5% (+13.7%) โดยไม่ต้องเปลี่ยนโมเดล

The Blueprint: สถาปัตยกรรมแบบ Decoupled

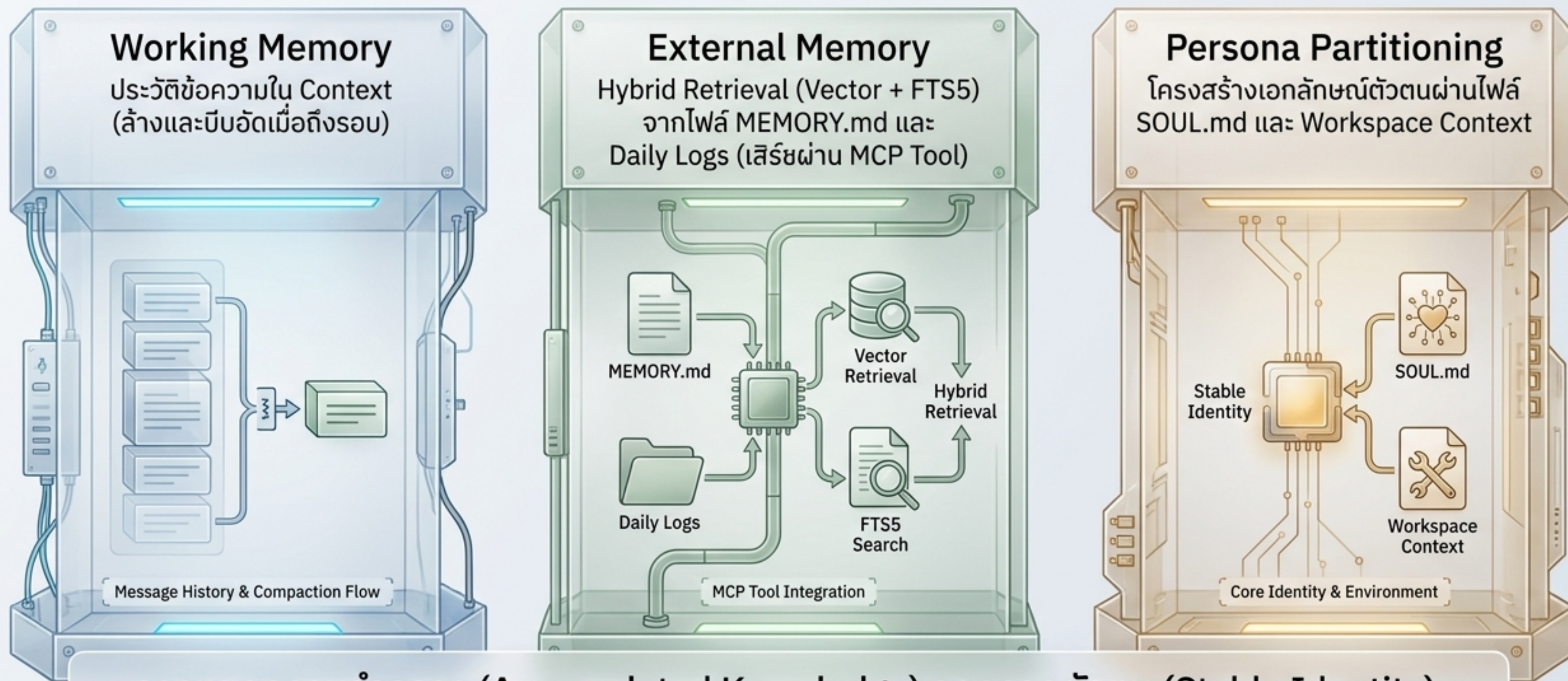


การแยก Core และ Harness ออกจากกัน ทำให้เราสามารถอัปเดตระบบ Runtime ได้โดยไม่กระทบกับตรรกะของแอปพลิเคชัน

บริหาร Context Window ด้วย Semantic Compaction



สถาปัตยกรรมความจำ 3 ระดับ (3-Tier Context Architecture)



การแยกความจำถาวร (Accumulated Knowledge) ออกจากตัวตน (Stable Identity) คือคุณสมบัติสำคัญที่ป้องกันไม่ให้ AI พฤติกรรมเพี้ยนเมื่อข้อมูลเพิ่มขึ้น

Memory Injection Comparison

Diagnostic Matrix	Working Memory (Compaction)	External Memory (Hybrid)	Structured Context (Persona)
Lifecycle	1 Session (Perishable)	Persistent (Cross-session)	Static/Curated (Lifetime)
Access Pattern	In-context (Immediate)	On-demand (Agent-initiated Tool Call)	Pre-retrieval Injection (Session Start/Post-compaction)
Governance	Event-driven Compression (sema-code-core)	3-Level Degradation (Vector -> FTS5 -> Keyword)	Workspace-scoped File System (SOUL.md)

สถาปัตยกรรมนี้เปลี่ยน Context จาก ถึงขยะรวมข้อมูล เป็น ทรัพยากรที่ถูกคัดกรองอย่างเป็นระบบ

The Capability Stack: นิเวศส่วนขยาย (Extension Ecosystem)

Layer 4: Execution Control (Hooks)

จุดเชื่อมต่อระดับ Harness เพื่อดักจับและควบคุมการทำงาน (เช่น Audit logs) โดยที่เอเจนต์ไม่รู้ตัว

Layer 3: Context Engineering (Skills)

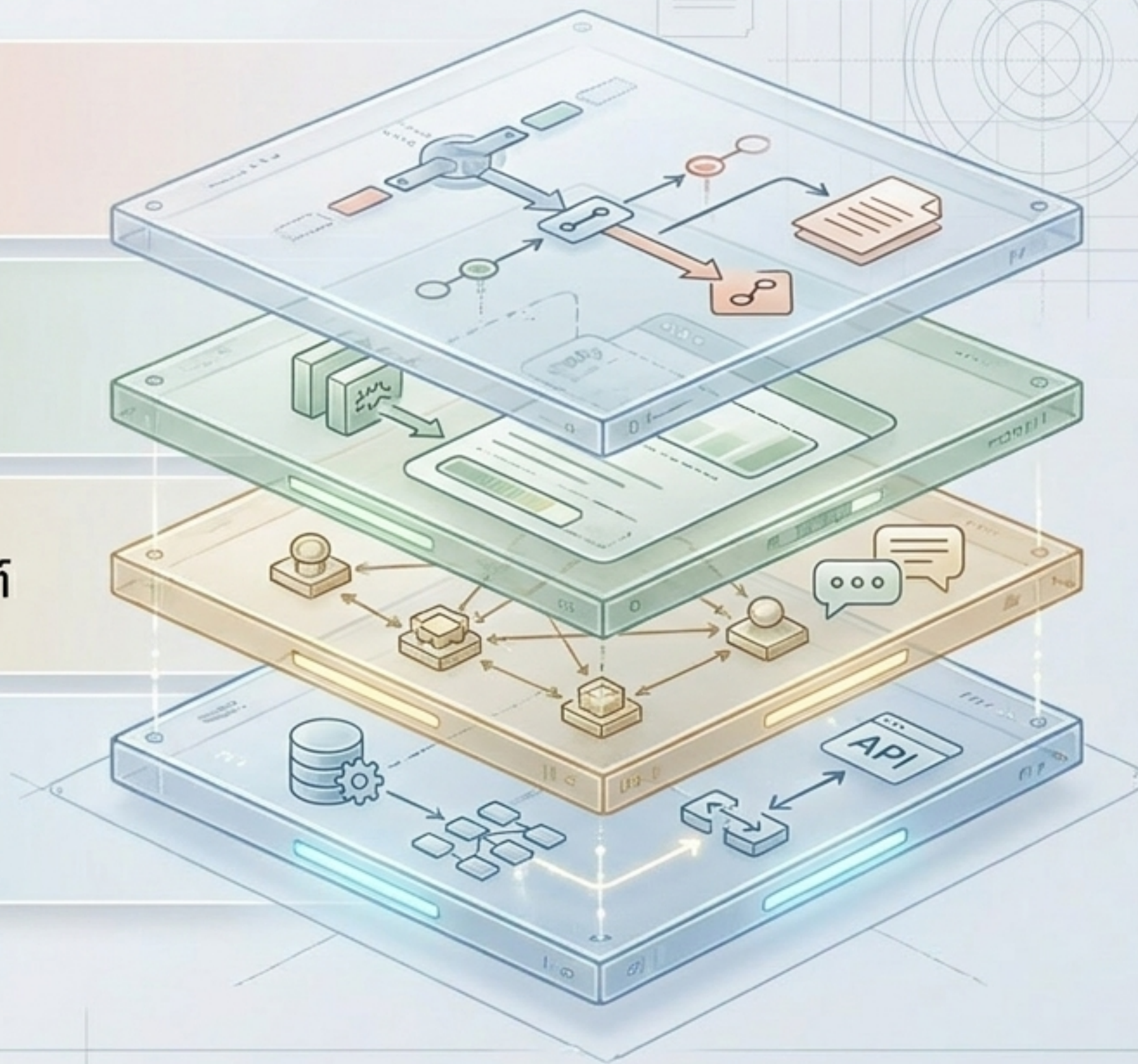
จัดการความสามารถเฉพาะทางแบบ Progressive Loading โดยไม่กินพื้นที่ Context Window เปล่าประโยชน์

Layer 2: Reasoning Scope (Subagents)

กระจายภาระทางความคิด (Cognitive Load) ไปยังเอเจนต์เฉพาะทาง - โต้ตอบผ่าน Natural Language Prompt

Layer 1: Action Space (MCP Tools)

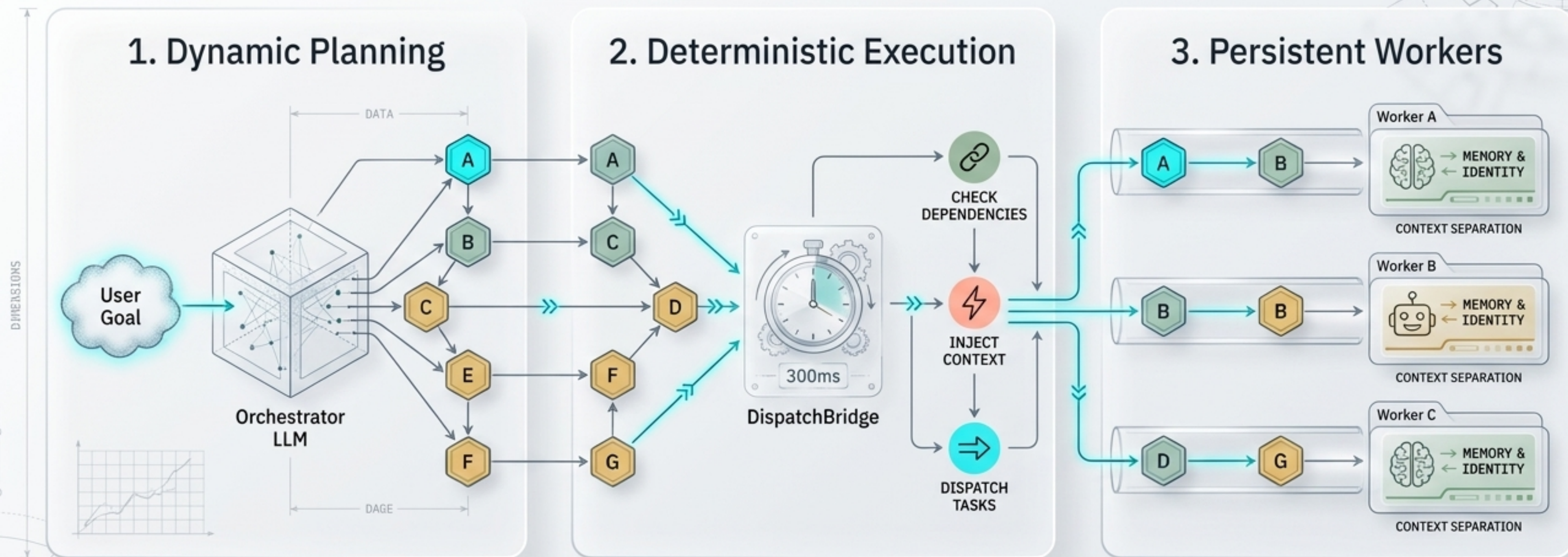
ขยายขีดความสามารถในการกระทำ (เช่น Database, API) - โหลดเป็น Typed Schema



Multi-Agent Orchestration Matrix

Models	Stateless Swarm (e.g., OpenAI Swarm)	Explicit Graph (e.g., LangGraph)	Dynamic Orchestrator (e.g., AutoGen)	DAG Teams (SemaClaw)
Flexibility	High	Low	High	High (LLM Plans dynamically)
Debuggability	Low	High	Low	High (Deterministic DAG)
State Persistence	None	High	Medium	High (Persistent Personas)
Failure Isolation	Poor	Excellent	Poor	Excellent (Node-level)

จาก ReAct สู่ DAG: แยกการวางแผนออกจากการทำงาน



Orchestrator LLM ย่อยเป้าหมายและสร้างกราฟโครงสร้างงาน (Directed Acyclic Graph) ก่อนเริ่มทำงานจริง

DispatchBridge (ทำงานทุก 300ms) ตรวจสอบ Dependencies และควบคุมการจ่ายงาน

งานถูกส่งให้ Worker ที่มี Persona และความจำของตัวเอง (แยก Context ชัดเจน ป้องกันการล้นของข้อมูล)

PermissionBridge: ระบบรักษาความปลอดภัยแบบ Human-in-the-Loop

Gate 1:
Internal Tools
(Pre-authorized)

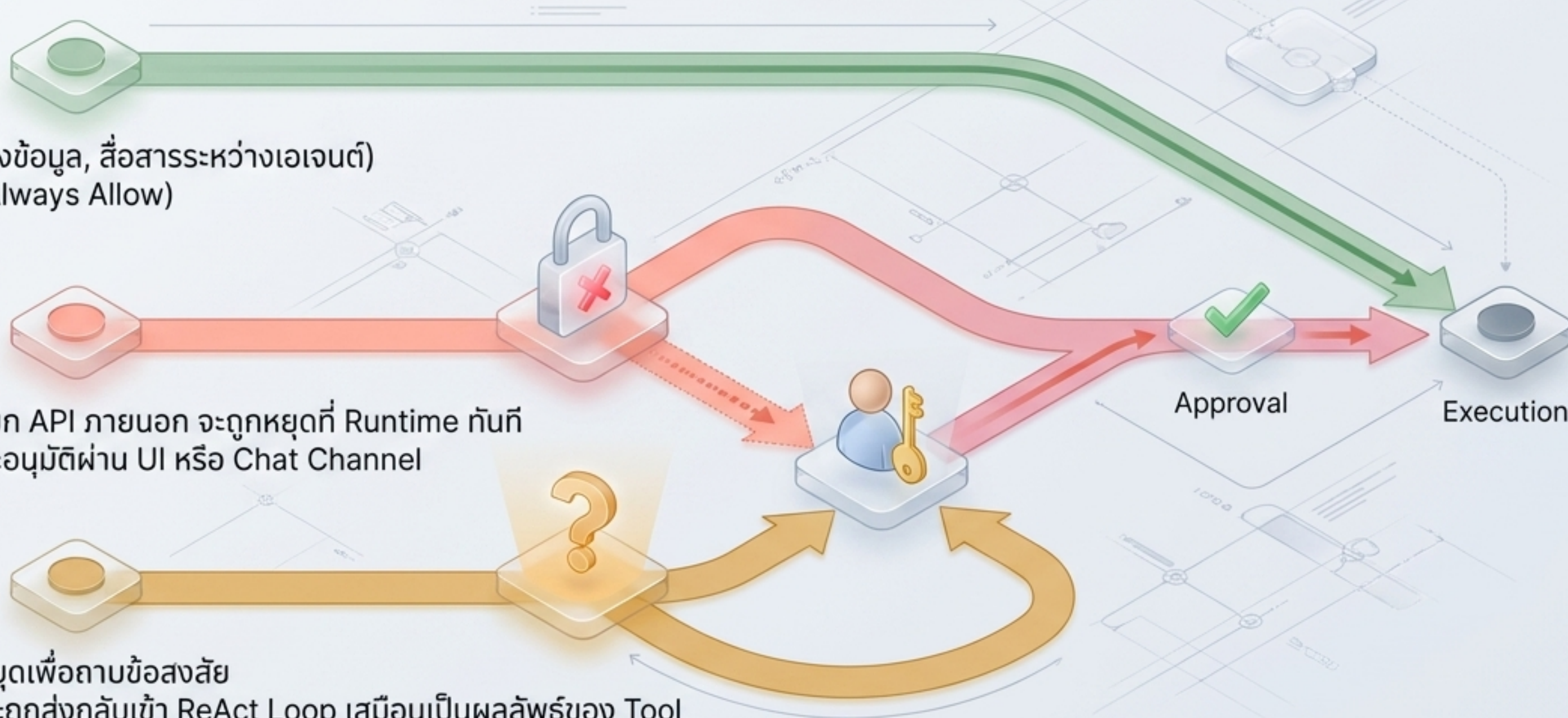
เครื่องมือระบบ (ดึงข้อมูล, สื่อสารระหว่างเอเจนต์)
ปล่อยผ่านทันที (Always Allow)

Gate 2:
External Tools
(High Risk)

การเขียนไฟล์, เรียก API ภายนอก จะถูกหยุดที่ Runtime ทันที
รออนุญาตมนุษย์จะอนุมัติผ่าน UI หรือ Chat Channel

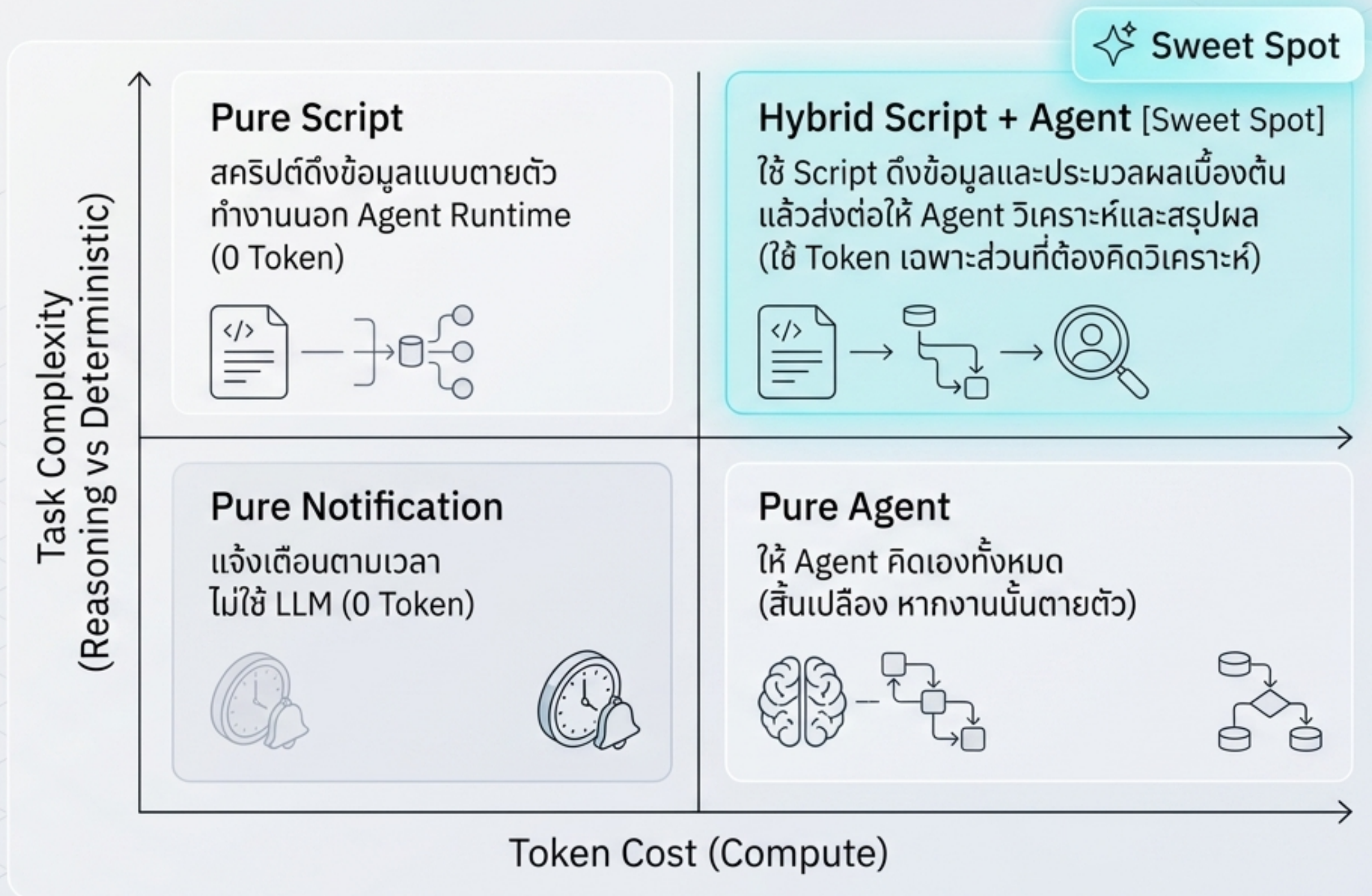
Gate 3:
Ask User Tool

เอเจนต์สามารถหยุดเพื่อถามข้อสงสัย
ข้อมูลจากมนุษย์จะถูกส่งกลับเข้า ReAct Loop เสมือนเป็นผลลัพธ์ของ Tool



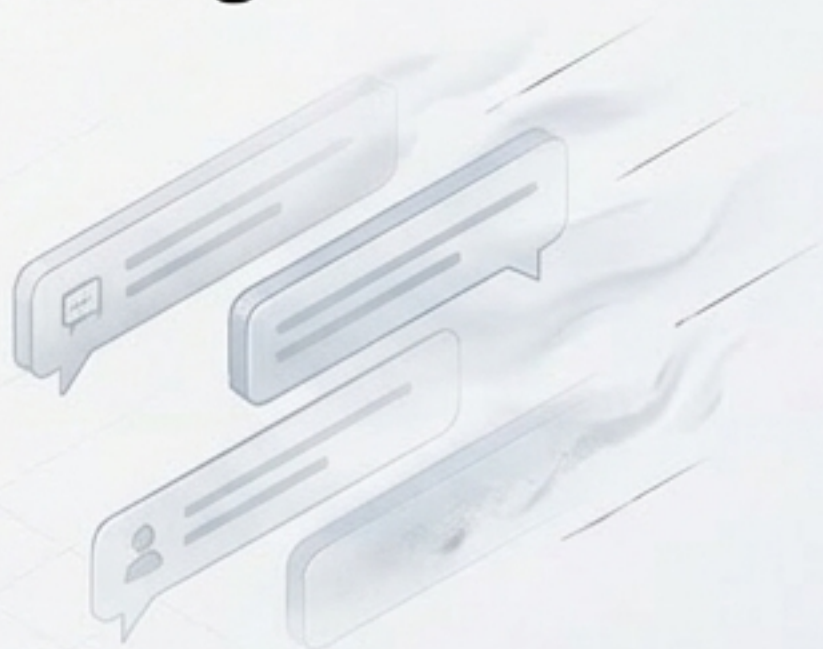
การอนุมัติ (Authorization) คือ First-class Primitive ระดับ Runtime ไม่ใช่แค่ตัวกรองระดับแอปพลิเคชัน

Execution Modes: การใช้ Token ให้คุ้มค่ากับความซับซ้อน



Knowledge Sedimentation: จาก Vibe Working สู่ Vibe Learning

Vibe Working



ปัญหาของ Context ที่วุ่นวาย:

ความรู้ที่ได้จากการแก้ปัญหาจะถูกกลบหายไปเมื่อหมดเซสชัน หรือถูกฝังกลบใน Log ที่ค้นหายาก

Vibe Learning



- **Wiki Knowledge Base:** สถาปัตยกรรมของ SemaClaw บังคับให้เกิดการตกตะกอนความรู้ ลงในไฟล์ Plain Markdown
- **Human-Agent Co-authorship:** ผู้ใช้สามารถแก้ไขโครงสร้างได้โดยตรง และ Agent จะดึงความรู้นี้กลับมาใช้ในครั้งต่อไป

ความฉลาดที่แท้จริงคือความสามารถในการสะสมความรู้ (Intelligence Compounds)

The Substitution Principle: Harness แทนที่ Capability ได้อย่างไร?

Retrieval > Parametric Knowledge

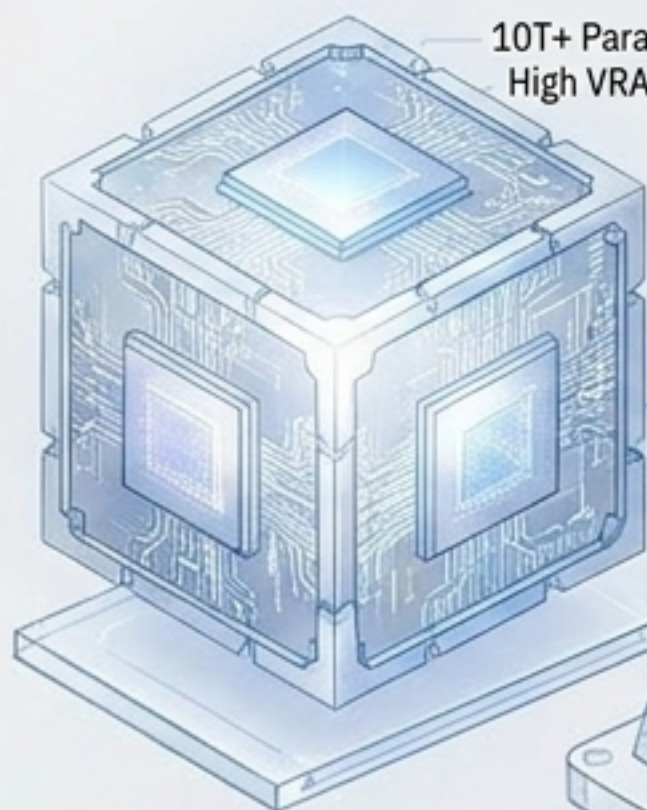
Structured Memory Architecture

การมีโครงสร้างความรู้ที่ดี ลดความจำเป็นที่โมเดลต้องจำทุกอย่างจาก Pre-training

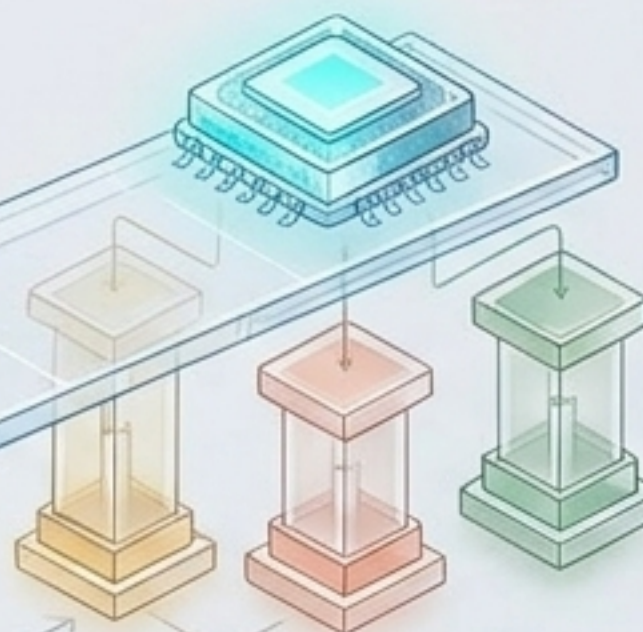


Frontier Model (Expensive & Massive)

10T+ Parameters
High VRAM Usage



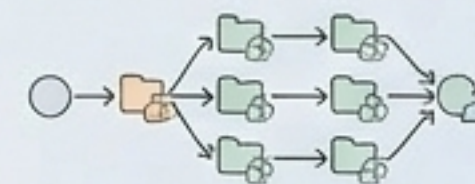
Small Model (SemaClaw Harness)



Decomposition > Massive Context Windows

DAG Task Automation

การชอยงาน (DAG Teams) ช่วยลดภาระ Context ทำให้ไม่ต้องพึ่งพาโมเดลที่มีหน้าต่างบริบทมหาศาล



Skills > Generalist Reasoning

Pre-loaded Tool & Skill Modules

การโหลด Skills เฉพาะทางล่วงหน้า ช่วยให้โมเดลขนาดเล็กทำงานเฉพาะเจาะจงได้เทียบเท่า Frontier Models



การลงทุนในสถาปัตยกรรม SemaClaw ทำให้ระบบ AI ของคุณ ถูกลง, เร็วขึ้น และ เชื่อถือได้มากกว่า การรออัปเดตโมเดลเวอร์ชันถัดไป

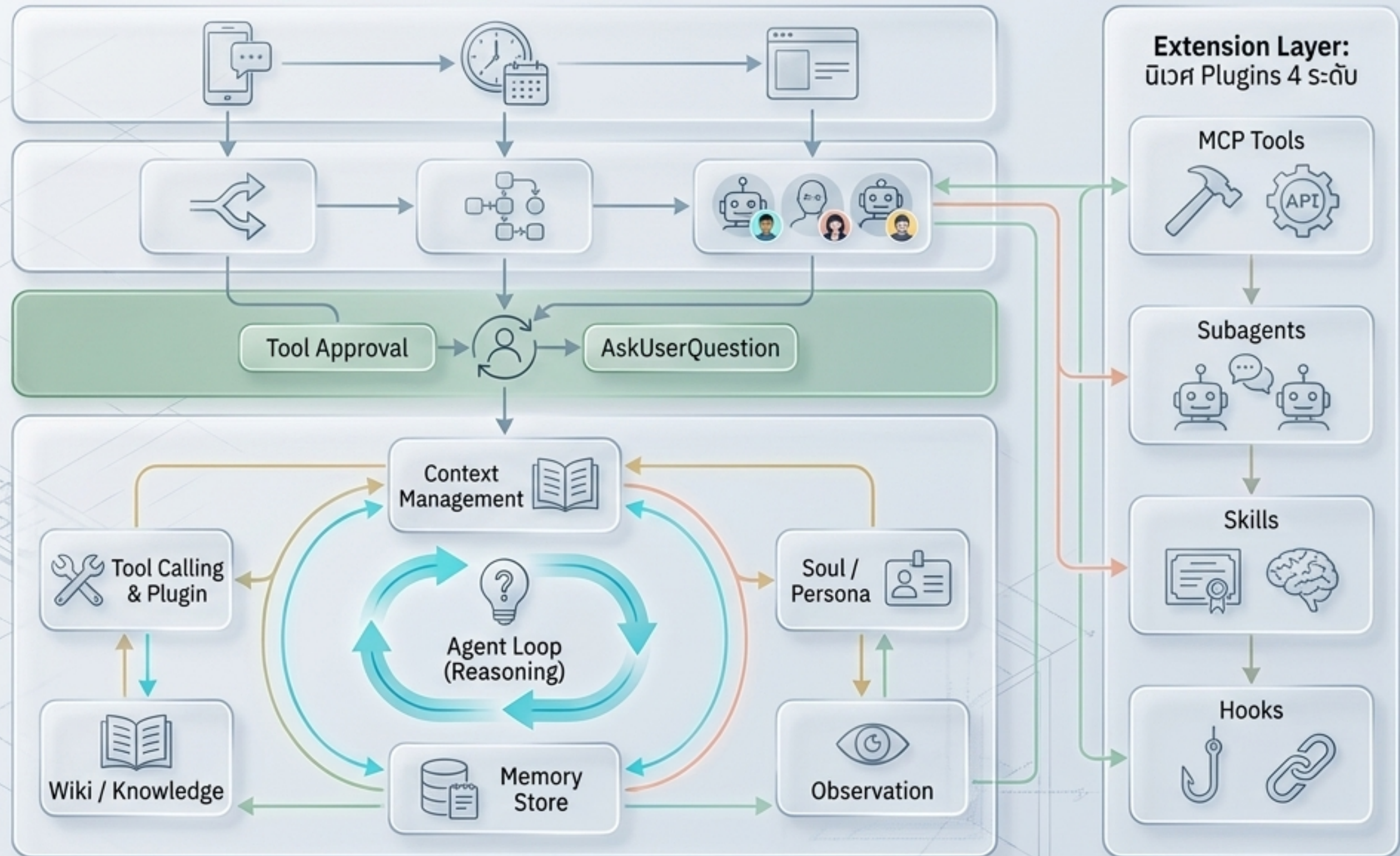
สถาปัตยกรรม: The Glassbox Engineered Blueprint

Input Layer:
Channels & Web UI

Orchestration Layer:
Message Router -> DAG Task
Dispatch -> Agent Teams

Safety Layer:
PermissionBridge (Tool
Approval / AskUserQuestion)

Core Execution:
ReAct Loop วนลูปด้วย
Context Management,
SOUL Persona, Memory
Store และ Wiki Knowledge



สถาปัตยกรรมแบบเปิดนี้ไม่ได้สร้างแค่ ผู้ช่วย (Assistant) แต่กำลังสร้าง
โครงสร้างพื้นฐานสำหรับความร่วมมือระยะยาว (Long-term Collaborative Infrastructure) ระหว่างมนุษย์และ AI